Optimization

Objectives, searches, and constraints

Open slides

Optimization is the task of choosing values that make an objective function as small or as large as possible. We start with an ordinary function that does not require statistics. Later, the same language will be used for least squares, likelihoods, and animal models.

The important idea for this course is practical:

- define a model,
- define the objective,
- search for the parameter values that optimize it,
- check whether the answer makes biological and statistical sense.

Formal Definition

Objective

Let θ denote the parameter vector and let Θ denote the parameter space, meaning the set of values that θ is allowed to take. An unconstrained minimization problem is written as

$$\hat{\theta} \in \arg\min_{\theta \in \Theta} f(\theta),$$

where $f(\theta)$ is the objective function. The notation arg min means "the argument value that gives the minimum". If there is only one minimum, $\hat{\theta}$ is a single point. If several values give the same minimum, the solution is a set.

The notation is compact, so define every symbol before using it:

Symbol	Meaning
θ	A candidate value, or vector of values, that we can choose. In statistics these are usually model parameters; in a plain optimization example they are decision variables.
Θ	The parameter space: all values that θ is allowed to take.
$\hat{\theta}$	The value of θ that solves the optimization problem. The hat marks the selected or estimated value.

Symbol	Meaning
$f(\theta)$	The objective function. It maps a candidate value θ to one number that we want to minimize or maximize.
$\arg\min$	"The argument that gives the minimum." The result is the input value θ, not the minimum value of $f(\theta)$ itself.
$\arg\max$	"The argument that gives the maximum."
$g_j(\theta)$	The jth inequality constraint.
$h_k(\theta)$	The kth equality constraint.
$j = 1, ..., m$	There are m inequality constraints.
$k = 1, ..., r$	There are r equality constraints.

A maximization problem is written as

$$\hat{\theta} \in \arg\max_{\theta \in \Theta} f(\theta).$$

Because minimizing $-f(\theta)$ is equivalent to maximizing $f(\theta)$, most software only needs one convention. In R, `optim()` minimizes by default:

$$\arg\max_{\theta \in \Theta} f(\theta) = \arg\min_{\theta \in \Theta} -f(\theta).$$

A constrained optimization problem adds restrictions to the parameter space:

$$\begin{aligned} \hat{\theta} &\in \arg\min_{\theta \in \Theta} f(\theta) \\ \text{subject to} \quad g_j(\theta) &\leq 0, \quad j = 1, ..., m \\ h_k(\theta) &= 0, \quad k = 1, ..., r. \end{aligned}$$

The functions $g_j(\theta)$ define inequality constraints and the functions $h_k(\theta)$ define equality constraints. In this course, we will mostly use simple bound constraints, for example $0 \leq w \leq 40$.

First Example: Fence A Paddock

Model

Suppose we have 80 meters of fencing and want to build a rectangular paddock. A rectangle has two widths and two lengths:

$$2w + 2\ell = 80.$$

Divide both sides by 2:

$$w + \ell = 40.$$

That is where the 40 comes from. If the width is w, the length must be

$$\ell(w) = 40 - w,$$

and the area is

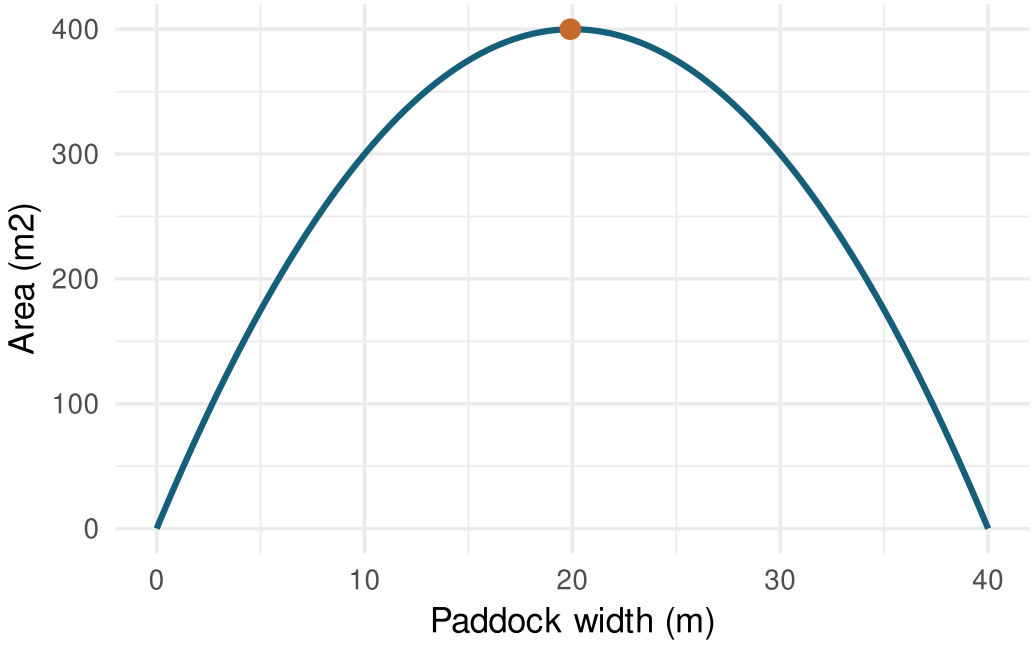
$$A(w) = w(40 - w).$$

The question is easy to state: which width gives the largest area?

```
fence_m <- 80

paddock_area <- function(width, fence_m = 80) {
  length <- fence_m / 2 - width
  width * length
}

candidate_widths <- seq(0, fence_m / 2, length.out = 200)

area_grid <- data.frame(
  width = candidate_widths
)
area_grid$length <- fence_m / 2 - area_grid$width
area_grid$area <- paddock_area(area_grid$width, fence_m = fence_m)

best_grid <- area_grid[which.max(area_grid$area), ]

kable(best_grid, digits = 2)
```

	width	length	area
100	19.9	20.1	399.99

```
ggplot(area_grid, aes(width, area)) +
  geom_line(color = "#155f7a", linewidth = 1.1) +
  geom_point(data = best_grid, color = "#c46a2b", size = 3) +
  labs(
    x = "Paddock width (m)",
    y = "Area (m2)"
  )
```



Search With `optim()`

Search

`optim()` minimizes by default. To maximize area, minimize the negative area:

$$\hat{w} = \arg\min_{0 \le w \le 40} [-A(w)].$$

```
negative_area <- function(width, fence_m = 80) {
  -paddock_area(width, fence_m = fence_m)
}

fit_area <- optim(
  par = 10,
  fn = negative_area,
  fence_m = fence_m,
  method = "L-BFGS-B",
  lower = 0,
  upper = fence_m / 2
)

kable(
  data.frame(
    method = c("grid search", "optim"),
    width = c(best_grid$width, fit_area$par),
```

```
    area = c(best_grid$area, -fit_area$value)
  ),
  digits = 3
)
```

method	width	area
grid search	19.899	399.99
optim	20.000	400.00

Bound Constraints

Constraints

The bounds are not technical decoration. They are part of the problem. A width below 0 meters or above 40 meters is impossible.

$$0 \leq w \leq 40.$$

```
fit_tight_area <- optim(
  par = 10,
  fn = negative_area,
  fence_m = fence_m,
  method = "L-BFGS-B",
  lower = 0,
  upper = 15
)

kable(
  data.frame(
    constraint = c("correct bounds", "too-tight upper bound"),
    lower = c(0, 0),
    upper = c(fence_m / 2, 15),
    width = c(fit_area$par, fit_tight_area$par),
    area = c(-fit_area$value, -fit_tight_area$value)
  ),
  digits = 3
)
```

constraint	lower	upper	width	area
correct bounds	0	40	20	400
too-tight upper bound	0	15	15	375

When the answer lands exactly on a boundary, the boundary is part of the result and needs interpretation.

Where This Goes Next

Model

The paddock example used a single decision variable, w, and a directly readable objective, $A(w)$. The next chapters use the same structure with statistical objectives:

Next topic	What gets optimized
Least squares	A line is chosen by minimizing squared vertical distances to observed points.
Maximum likelihood	Parameter values are chosen by maximizing the probability of the observed data under a model.
Animal models	Fixed and random effects are chosen by balancing fit to records against shrinkage from covariance assumptions.

The mathematical form changes, but the workflow does not: define the allowed values, define the objective, search for the optimum, and interpret the result.

Exercises

1. Change `fence_m` in the paddock example. What happens to the optimum width?
2. Change the starting value in `fit_area`. Does the solution change?
3. Change the upper bound in `fit_tight_area`. At what point does the answer become boundary-driven?
4. Rewrite the area objective for a rectangle where one side lies along an existing wall, so only three sides require fencing.
5. Write a minimization problem for choosing a width that gets as close as possible to a target area of 300 m2.